Free Plugin Brings Machine-Checked Formal Specifications to AI Coding Assistants

Z Spec for Claude Code generates, type-checks, and animates ISO-standard Z specifications —

so developers can build new systems without bugs, derive exhaustive tests for existing ones, and find defects that testing alone cannot reach

Press Release

SAN FRANCISCO — August 2026 — Punt Labs today released Z Spec, a free Claude Code plugin that brings formal mathematical specifications to AI-assisted software development. Unlike natural-language specification tools that produce documents humans interpret, Z Spec generates specifications in Z notation — an ISO standard (13568:2002) backed by 45 years of academic research [1, 2]. fuzz machine-checks them for internal consistency; ProB explores them for reachable states. Z Spec supports three workflows: writing the spec first to create new systems without bugs, deriving exhaustive test suites for existing code, and modeling running systems to find defects that testing alone cannot reach (see FAQ 2).

Problem

Software teams face three interconnected problems that testing alone cannot solve.

New systems start without a blueprint. Developers using AI coding assistants generate code from natural-language prompts — what practitioners call "vibe coding" [3]. The AI produces plausible-looking implementations, but each generation makes dozens of unstated assumptions about state transitions, boundary conditions, and invariants. A developer who prompts "add photo sharing to my app" receives code that works for the demo. Whether it enforces size limits, handles concurrent uploads, or prevents unauthorized deletion depends on assumptions the AI made and the developer never reviewed. Without a precise specification written *before* the code, there is no way to know what the system should do — let alone whether the code does it.

Existing code has shallow test coverage. A test suite with 95% line coverage still misses the state combination where a user deletes a photo while another user is mid-upload. Developers write tests for the cases they thought of; the cases they did not think of remain untested. Formal specifications catch these gaps mathematically: an invariant like `owner(photo) = uploader` generates test partitions that cover every equivalence class, not just the inputs a developer happened to imagine.

Running systems harbor latent defects. Even well-tested code accumulates state assumptions that were never made explicit. A system that worked correctly for three years fails when a new feature interacts with an invariant nobody documented. Modeling the existing system as a formal specification surfaces these hidden assumptions and lets a model-checker explore states the test suite never reached.

The methods to solve all three problems have always worked. In a survey of 62 industrial formal-methods projects, 92% reported quality increases and 0% reported decreases [4]. But writing formal specifications by hand required hours of skilled effort per schema — a cost that confined them to safety-critical domains like avionics and railways.

Solution

Z Spec collapses the cost of formal specification and supports three workflows that cover the full software lifecycle.

Spec-first development. Before writing a line of code, a developer describes the new system in natural language and runs `/z-spec:code2model` with a description instead of a codebase path. The LLM drafts a Z specification with states, invariants, and operations. The developer runs `/z-spec:check` to type-check the spec with fuzz and `/z-spec:test` to animate it with ProB, iterating until the specification captures the intended behavior exactly. The spec then serves as the blueprint: `/z-spec:model2code` (Feature 7) generates implementation code and tests that conform to it, and `/z-spec:contracts` (Feature 6) produces runtime assertions that enforce invariants in production. The code is correct by construction, not by accident.

Test coverage for existing code. For systems that already work — or seem to — a developer runs `/z-spec:code2model` to generate a specification from the codebase. Then `/z-spec:partition` (Feature 5) applies the Test Template Framework to derive exhaustive test cases from the spec's mathematics, and `/z-spec:contracts` generates runtime assertions. The spec exposes equivalence classes and boundary conditions that the developer never thought to test — even for code with no known bugs.

Bug finding in existing systems. The same `/z-spec:code2model` workflow surfaces defects directly. If ProB finds a counter-example — a reachable state that violates an invariant — it reports the exact trace. The `/z-spec:audit` command (Feature 8) automates the full chain: model, check, test, and report. For existing code in production, this is how developers find bugs that testing alone misses.

Across all three workflows, the specification is the single source of truth. Tests, proofs (Feature 9), and contracts are all derived from it, not maintained independently.

Customer Quote

> *"I used Z Spec two different ways on the same project. For the new payment module, I wrote the spec first — before any code. ProB found three states I hadn't constrained, and I fixed them in the spec instead of debugging production. For the existing authentication flow, I ran `/z-spec:code2model` on code we'd shipped a year ago, and `/z-spec:partition` generated forty test cases. Twelve of them tested boundaries we'd never covered. Two of those caught real bugs."*
>
> — **Marta Reyes**, Senior iOS Developer, Fitness Startup (6-person team)

Getting Started

Getting started takes three commands. First, install:

```
claude plugin install z-spec@punt-labs
```

Then set up the verification tools (one time only):

```
/z-spec:setup all
```

Then choose a workflow with `/z-spec:code2model`: describe a new system in natural language to write its spec first, or point it at your codebase to generate one from existing code. Within five minutes, a developer can generate their first specification, type-check it, and animate its state space. No formal-methods background is required — the LLM translates system descriptions into Z, and fuzz and ProB do the mathematical heavy lifting.

Spokesperson Quote

> "Formal methods have a 45-year track record, but they've been locked behind a skill barrier that most developers couldn't justify crossing. We built Z Spec because AI coding assistants have eliminated that barrier — the LLM drafts the spec, and established tools check it. The insight is that the same specification serves three jobs: it's a blueprint before you code, a test generator after you code, and a bug finder for code you shipped years ago. The math hasn't changed. The cost has."
>
> — **J Freeman**, Punt Labs

Call to Action

Z Spec is available now at `https://github.com/punt-labs/z-spec` under the MIT License. Install via the Punt Labs plugin marketplace or the one-line installer in the README. Free, open source, no account required.

Frequently Asked Questions

External FAQs

Q1: What is Z Spec and who is it for?

Z Spec is a Claude Code plugin for developers who build stateful systems — anything with modes, invariants, or state transitions (authentication flows, shopping carts, protocol handlers, device controllers). It supports three workflows. (1) Spec-first development: the developer writes a formal specification before coding and generates implementation from it. (2) Test coverage derivation: a specification of existing code produces exhaustive test cases and runtime contracts. (3) Bug finding: modeling an existing system surfaces invariant violations that testing missed.

Q2: How is this different from GitHub Spec Kit, Amazon Kiro, or other spec tools?

Those tools generate natural-language specifications — structured prose that humans read and interpret. Z Spec generates *machine-checkable* specifications in Z notation [1]. When fuzz accepts a spec, its type system guarantees internal consistency. When ProB animates it, the tool explores every reachable state and reports any invariant violation with an exact counter-example trace.

Natural-language specs can be ambiguous: "the system should handle concurrent access" has multiple valid interpretations. A Z invariant like $\forall u : \mathit{ActiveUsers} \bullet \mathit{session}(u) \neq \emptyset$ has exactly one. The spec is the contract; the checker is the enforcer.

The difference compounds across workflows. A natural-language spec cannot generate test partitions (what inputs would you derive from prose?), cannot produce runtime contracts (which predicate do you assert?), and cannot serve as a blueprint for code generation (the implementation would have to re-interpret the ambiguity). A Z specification does all three because it is mathematical, not linguistic.

Research confirms this distinction matters. When formal specifications serve as machine-checkable contracts verified by a formal tool, LLM-generated code achieves higher correctness rates [5, 6]. When the LLM verifies its own output against natural-language specs, it fails systematically [7].

Q3: How do I get started?

Install the plugin, run `/z-spec:setup all` to install fuzz and ProB, then choose a workflow. For a new system, run `/z-spec:code2model` with a natural-language description to write the spec first. For existing code, run `/z-spec:code2model` pointing at your codebase, then `/z-spec:partition` to derive test cases or `/z-spec:test` to find bugs.

Q4: How much does it cost?

Free. MIT License. No account, no API key, no usage limits. The plugin orchestrates open-source tools (fuzz, ProB) that run locally on your machine.

Q5: Do I need to learn Z notation to use this?

No. The LLM generates the specification and explains each schema in plain English. You can review and iterate on the spec in natural language ("add a constraint that prevents negative balances") and the plugin translates your intent into Z. Over time, reading Z becomes natural — the notation is concise and mathematically precise. The plugin includes a notation reference (`/z-spec:help`) for when you want to read or edit specs directly.

Q6: When should I use spec-first vs. modeling existing code?

New system with no code yet: Use `/z-spec:code2model` with a natural-language description to write the spec first, then generate code and tests once it captures your intended behavior. This is the highest-value workflow — bugs are cheapest to fix when no code exists yet.

Existing code you want to harden: Use `/z-spec:code2model` to generate a spec from the codebase, then `/z-spec:partition` to derive test cases and `/z-spec:contracts` to generate runtime assertions. This is valuable even when the code has no known bugs — the spec reveals untested boundaries.

Existing code you suspect has bugs: Use `/z-spec:code2model` followed by `/z-spec:test`. If ProB finds a counter-example, it reports the exact state trace. The `/z-spec:audit` command automates the full chain for rapid analysis.

In practice, developers often use all three workflows on different parts of the same project.

Q7: What happens to my data?

Everything runs locally. Specifications are LaTeX files in your project directory. fuzz and ProB execute on your machine. The only network traffic is between Claude Code and the Claude API (which you already use). Z Spec adds no telemetry, no external services, no data collection.

Internal FAQs

Value & Market

Q8: What is the total addressable market?

The primary market is developers using AI coding assistants who build stateful systems.
AI-assisted developers: 84% of developers use or plan to use AI coding tools [8], up from 76% in 2024 [9]. Estimates of the global professional developer population range from 20–37 million [10]. Applying the 84% rate gives roughly 17–31 million developers using AI assistance.
Stateful system builders: Not every project needs formal specs. The natural fit is systems with state machines, invariants, protocols, or concurrent access. We do not have data on what percentage of projects this represents. The range could be narrow — perhaps 5% of developers who already prioritize correctness — or broad, reaching 40%+ of developers who have been burned by a state bug. Our validation (FAQ 11) will test where reality falls.
Claude Code users specifically: Z Spec is a Claude Code plugin, so the immediate addressable market is Claude Code's developer base. As a free plugin in an emerging ecosystem, the initial target is hundreds to low thousands of active users in the first year.
This is a bottoms-up estimate with wide uncertainty at the "stateful systems" layer. The adoption rate and developer population are grounded in survey data; the filter for who needs formal specs is not.

Q9: What evidence do we have that customers want this?

Internal adoption is real and independently verifiable, but it is not a clean market signal — three of the largest consumers now mandate the practice.
Positive signals:

- Eight of the sibling projects in the Punt Labs portfolio have reached for Z Spec to model a real subsystem. Counting only real specifications — excluding z-spec's own dogfood corpus of examples, tutorials, and test fixtures, and excluding `.tmp/` scratch clones and `.claude/worktrees/` duplicates in the sibling repositories — there are 57 specification files across those 8 repositories. The breakdown: biff (10), lux (26, the largest consumer), vox (12), quarry (2), ethos (2), beadle (2), koch-trainer-swift (2), and mcp-proxy (1). Several of these are variant families investigating one underlying subject rather than 57 independent subjects — lux alone has five subjects (board ordering, display-crash loops, header-toggle reconciliation, hub-display reconciliation, scene-ID namespacing) spread across 15 files, and biff's NATS-relay work spans 3 files under one subject. The distinct-subsystem count is therefore in the low forties, not 57.
- The Koch Trainer iOS app was specified before implementation (spec-first): ProB validated the state machine before a line of Swift was written. Biff's own committed `.audit.json` report is a real artifact of the test-derivation workflow, generated by `/z-spec:audit` against its connection-lifecycle spec. And several of lux's reconciliation specifications carry deliberate "buggy" variants alongside the fixed version: a spec that cannot reproduce the defect it guards against is too abstract to trust. So the team builds the failing model first, then confirms the passing one catches what the failing one doesn't. That is bug finding with a falsification test built in, not just a claim that the spec "covers" the bug.
- All three of the largest consumers — biff, vox, and lux — began writing Z specifications independently in early March 2026. Four months later, on 2026-07-04, all three repositories made the practice mandatory in one coordinated policy decision, not three separate ones.

Roughly two-thirds of the 57 specification files were written after that date, so most of the corpus reflects a requirement rather than open-ended choice. But the four months of voluntary use that preceded it justified making it policy.

- An external contributor (Eric Bowman) independently forked the project and added four commands (`/z-spec:prove`, `/z-spec:contracts`, `/z-spec:oracle`, `/z-spec:refine`), indicating perceived value in the derived-artifacts workflow beyond the original author.
- The broader "spec-driven development" category is attracting commercial investment: GitHub Spec Kit, Amazon Kiro, and Tessl all launched in 2025–2026 [3].

This is evidence the mechanism works across genuinely different problem domains — a display server's reconciliation logic, a message relay's session lifecycle, a TTS daemon's call state machine — and that engineers found it valuable enough, unprompted, to justify making it policy four months later. It is not evidence of external adoption: every one of these specifications was written inside the same organization that builds Z Spec, largely by engineers or dispatched sub-agents with direct access to its creators. FAQ 11 is what still needs to happen before this hypothesis is validated in the market sense.

What we do not know:

- Whether developers outside Punt Labs' own portfolio would adopt the tool voluntarily, absent any mandate.
- Whether the formal-notation aspect is a draw ("finally, machine-checked rigor") or a deterrent ("I don't want to look at math symbols").
- Which workflow is the primary adoption driver — spec-first for new projects, test derivation for existing ones, or bug finding for legacy systems.
- Whether developers would use the upstream pipeline (prfaq → use-cases → z-spec) or treat Z Spec as a standalone tool.

Q10: Who are the competitors and why will we win?

Natural-language spec tools (GitHub Spec Kit, Amazon Kiro): Generate structured prose specifications. Strength: familiar syntax, low adoption barrier. Weakness: specs cannot be machine-checked for consistency or animated for state-space exploration. A natural-language spec that says "handle concurrent access" does not tell you *how* or catch the case where you forgot.

Formal verification research tools (Dafny, Alloy, TLA+, Lean 4): Provide machine-checked rigor. Strength: mathematical guarantees. Weakness: require the developer to write formal specifications by hand, which is time-consuming and requires specialized training. Not integrated into AI coding workflows.

Z Spec's position: The LLM eliminates the writing cost of formal specs. The established tools (fuzz, ProB) provide the rigor. This combination — AI-generated specs verified by formal tools — is the architecture that research identifies as productive [5, 11, 12]. Z is the formalism because it is an ISO standard with a mature type-checker and animator, not a research prototype. Z Spec covers the full lifecycle — spec-first development, test derivation, and bug finding — where competitors address only one phase.

Funded competitors (Tessl): Raised $125M (Series A, Index Ventures and Accel) for spec-driven development infrastructure. Focus is natural-language specifications, not formal verification. Strength: capital and enterprise distribution. Weakness: same ambiguity limitation as GitHub Spec Kit and Amazon Kiro — structured prose cannot be machine-checked.

Competitive response: The category is attracting capital ($125M from Tessl alone) and distribution advantages (GitHub/Microsoft, Amazon/AWS). If these players moved toward formal verification, they could integrate existing tools (Dafny, Alloy, or even Z/ProB) faster than 6–12

months. Our differentiation is formal-verification depth (21 commands, three workflows from one specification) in a niche that large players are unlikely to prioritize near-term. The structural moat is thin — the defensible value is in execution quality and being first to demonstrate the workflow.

Q11: What is our next step to validate the vision?

Internal dogfooding already shows the mechanism works (FAQ 9), including specifications that formally pin down real defects and prove a fix closes them. That validates feasibility and utility within one organization's own engineering practice. It does not validate market demand — adoption in the three largest consumers preceded a mandate that now requires it, and every one of these specifications was written by engineers or dispatched sub-agents with direct access to the tool's creators.

The remaining validation step is external: ship the plugin publicly and measure organic adoption outside Punt Labs. Specifically:

1. Release v1.0 on the Punt Labs marketplace (already at v0.20.2, shipped end-to-end on PyPI and via the plugin marketplace).
2. Track installs, active usage (`check`, `code2model`, `partition`, and `test` invocations), and GitHub stars over 90 days. Track which workflow each user enters through.
3. Interview 5–10 developers who try the plugin to understand: what triggered them to try it, which workflow they used (spec-first, test coverage, or bug finding), whether they continued using it, and why or why not.
4. Decision threshold: if fewer than 50 developers use the tool in 90 days, the hypothesis that "AI makes formal methods accessible" is not validated for this market and format.

Technical

Q12: What are the major technical risks?

Risk 1: LLM-generated specs may be well-typed but wrong.
fuzz verifies that a specification is internally consistent (types check, references resolve). ProB verifies that invariants hold across reachable states. Neither tool verifies that the spec *matches the developer's intent*. An LLM could generate a spec that type-checks, animates, and describes the wrong system. Research confirms this concern: LLM-generated formal annotations are less reliable than hand-written ones [13], and LLMs struggle with complex real-world systems [14].
Mitigation: The workflow is designed as a feedback loop, not a one-shot generation. The developer reads the spec (annotated with plain-English explanations), reviews the ProB animation trace, and iterates. The `/z-spec:partition` command derives concrete test cases from the spec, giving the developer a second check: "do these test cases match what I expect?" The spec is a conversation artifact, not a black box.
Risk 2: Tool installation friction.
fuzz must be compiled from C source. ProB requires a Tcl/Tk runtime. On macOS, this means Xcode command-line tools and Homebrew dependencies. The `/z-spec:setup` command automates this, but compilation failures on unusual system configurations could block adoption.
Mitigation: The `/z-spec:doctor` command diagnoses installation issues. Pre-built binaries for fuzz would eliminate the compilation step (not yet available).
Platform support: macOS and Linux are supported. fuzz compiles from C on both; ProB provides pre-built binaries for both. Windows is not tested — ProB supports Windows but fuzz compilation requires a POSIX toolchain (WSL or Cygwin).

Q13: What dependencies exist?

- **fuzz** (Mike Spivey, Oxford) — Z type-checker. Open source, stable, maintained by its creator since 1988. Compiled from C source.
- **ProB/probcli** (HHU Düsseldorf) — animator and model-checker. Open source, actively maintained. Pre-built binaries for macOS and Linux.
- **Claude Code** — the host environment. Z Spec is a Claude Code plugin and cannot run outside it.
- **Lean 4** (optional) — for `/z-spec:prove`. Installed via elan, with Mathlib as a dependency (2 GB on first build).

No dependency on external APIs, cloud services, or proprietary tools beyond Claude Code itself.

Q14: What is the scaling story?

Z Spec runs locally, so the scaling constraint is not infrastructure — it is specification complexity. ProB's model-checking is bounded by the state space: a specification with three bounded integers and two enumerations explores thousands of states in seconds; a specification with ten unbounded variables may not terminate.

The critical skill is *finding the right slices*. A system with 200 state variables cannot be specified as a single flat schema. The developer must identify which subsystems have interesting invariants — the authentication state machine, the payment workflow, the UI coordination logic — and specify each one at the abstraction level where ProB can explore the full state space. This is analogous to choosing what to unit-test: you do not test every line, you test the boundaries that matter.

The plugin does not yet guide this decomposition. Multi-spec coordination (specifications that reference each other across modules) is on the roadmap (FAQ 15) but not shipped. Until then, the practical ceiling is single-module specifications with bounded state — matching the corpus demonstrated so far (FAQ 9): single-subsystem state machines and reconciliation logic, not whole-service models.

Performance and single-user hardware. A second scaling dimension is real: the underlying engine (fuzz and ProB) is designed to scale, but the RAM and CPU on a typical developer laptop become the binding constraint before specification complexity does. Some real specifications from internal projects run comfortably; larger state spaces stress the machine before ProB itself runs out of room. The hardware-to-scale equation at each level is not yet mapped, and a hosted evaluation deployment has not been tried at scale. This is called out under Feasibility in the Risk Assessment.

Q15: What is the estimated development timeline?

Z Spec is at v0.20.2 with 21 commands shipped, on PyPI as `punt-z-spec` and on the Punt Labs plugin marketplace. Development priorities in delivery order:

1. **Stabilize and document** — polish existing commands, improve error messages, write tutorials for all three workflows (spec-first, test coverage, bug finding). This unblocks public adoption.
2. **Tutor mode** — adaptive learning that teaches Z notation as the developer uses the tool. Reduces the adoption barrier identified in the risk assessment.
3. **Pipeline integration** — structured handoff from the `use-cases` plugin, so use-case scenarios feed directly into Z Spec as specification seeds (FAQ 19).
4. **B-Method maturation** — move B commands from alpha to stable. Enables the deterministic refinement path (spec to proven-correct code).

5. **Multi-spec coordination** — specifications that reference each other across modules. Required for systems larger than a single file.

If scope compresses, items 3–5 are cut first. Item 1 is the minimum for public release. Item 2 directly addresses the highest-rated risk (adoption barrier).

Business

Q16: What is the revenue model?

Free and open source. Z Spec sits in the Punt Labs ecosystem of developer tools (Quarry, Biff, Vox, Lux) — none of which are monetized today. The plugin serves three strategic purposes:

1. **Ecosystem growth** — every Z Spec user installs the Punt Labs marketplace, increasing exposure to paid tools. Z Spec also anchors the prfaq → use-cases → z-spec pipeline (FAQ 19), creating pull-through for the other plugins.
2. **Proof of methodology** — Z Spec demonstrates spec-driven development concretely, validating the approach for potential application in commercial projects.
3. **Standards contribution** — formal methods accessibility benefits the broader developer community. This is a genuine motivation, not a rationalization.

There is no plan to charge for Z Spec. If the tool is successful, the revenue path is through commercial tools that benefit from the same audience, not through monetizing the plugin itself.

Q17: What are the key metrics for success?

Metric	Target (90 days)	Why it matters
Plugin installs	100+	Basic reach — are developers finding it?
Active users	50+	Adoption — used check or test beyond install?
Repeat users (used 3+ times)	15+	Retention — is the value sustained?
External contributions	5+	Community signal — do others invest in it?
Non-author specs generated	10+	Generalizability — works beyond our codebases?

Decision threshold: if active users are below 25 after 90 days, revisit whether the adoption barrier is too high or the value proposition is too narrow.

Q18: Why now? What has changed?

Two shifts converged:

AI coding assistants reached mainstream adoption (FAQ 8). Yet only 29% trust the accuracy of AI-generated output [8]. This gap — widespread use with low trust — created the "vibe coding" problem at scale: developers generating code from loose prompts with no systematic verification beyond testing.

LLMs became capable of drafting formal specifications. Research in 2024–2026 demonstrated that LLMs can generate correct formal annotations in Dafny (98.2% with feedback loop [11]), repair buggy Alloy specifications, and produce properties that find real vulnerabilities [15]. The key finding across this literature is that the productive architecture is LLM generation verified by a formal tool — not the LLM verifying itself [7, 12].

Z notation is an ISO standard with a mature type-checker (fuzz, since 1988) and a mature animator (ProB, since 2003). These tools existed before LLMs. What was missing was an accessible way to *write* the specifications. The LLM provides that. No 2023–2026 paper has

studied Z specifically with LLMs [16] — this is an open gap, not a validated approach. Z Spec is an early tool to explore it; we do not claim to be the first.

Q19: How does Z Spec fit into the broader Punt Labs pipeline?

Z Spec is designed as the third stage of a rigorous product-to-specification pipeline:
prfaq (product thinking) → **use-cases** (structured requirements) → **z-spec** (formal specification)
The `prfaq` plugin guides Working Backwards product discovery, producing a decision document that defines the customer, problem, and solution. The `use-cases` plugin (in development) guides Jacobson-Cockburn structured use cases — actors, goals, basic scenarios, and extensions — that translate the product vision into concrete behavioral requirements. Z Spec then formalizes those requirements as machine-checkable specifications with invariants, state transitions, and operations.
Taken together, the pipeline offers rigor at every level: product rigor (is this worth building?), requirements rigor (what exactly should it do?), and specification rigor (does the math hold?). Each tool can be used independently, but the pipeline matters most for new projects, where spec-first development starts from a clear product vision and precise requirements.
This pipeline is a hypothesis. The `use-cases` plugin is in early development, and no user has yet walked the full prfaq → use-cases → z-spec path outside the author's own projects.

Risk Assessment

Risk	Rating	Assessment
Value	Medium	Three problems are real and growing: new systems built without blueprints, existing code with shallow test coverage, and running systems with latent defects. Internal usage across 8 sibling repositories (57 specification files, FAQ 9) shows the mechanism delivers real value — including formally proving a concurrency fix closes the exact defect a weaker design would not, with a falsification-test variant confirming it — but external developer adoption, and which workflow drives it, remains unproven. The math-notation barrier may limit adoption to developers who already value correctness.
Usability	Medium	Three-command onboarding is simple, but tool installation (compiling fuzz from source, Tcl/Tk for ProB) creates friction. Z notation is unfamiliar to most developers, even with LLM-generated explanations. The tutor mode (FAQ 15) is the primary mitigation.
Feasibility	Medium	The product exists at v0.20.2 with 21 commands, on PyPI and the plugin marketplace. fuzz and ProB are proven tools, so the core mathematics is not in question, and 57 real specification files across 8 sibling repositories (FAQ 9) show the pipeline holds up across genuinely different problem domains, not just the tool's own examples — including deliberate falsification-test variants that confirm a spec actually catches the defect it claims to model. Two feasibility concerns are still live: (a) LLM spec quality (FAQ 12) — a well-typed spec can still describe the wrong system; and (b) performance on a single-user machine (FAQ 14) — the engine scales, but RAM and CPU on a typical developer laptop bind before ProB does, and the hardware-to-scale equation at each specification size is not yet mapped. Stability work is ongoing; the deployment story beyond a personal laptop has not been evaluated.
Viability	Low	Free, open-source plugin with no infrastructure costs. Strategic value to the Punt Labs ecosystem. No regulatory, legal, or brand risks. The only viability question is opportunity cost — is this the best use of development time?

Riskiest assumption: Developers using AI coding assistants will voluntarily adopt a tool that introduces mathematical notation into their workflow, even when the LLM mediates the notation. The validation plan (FAQ 11) tests this directly.

Feature Appendix

Defines the scope boundary for Z Spec. Features are categorized to prevent scope creep and force prioritization.

Must Do

Features essential for the three core workflows. Without these, the product does not solve the problem.

- **F1.** **Spec generation** — `/z-spec:code2model` — the LLM drafts Z specifications from a natural-language description (spec-first development) or from codebase analysis (test derivation and bug finding for existing systems)
- **F2.** **Type-checking with fuzz** — `/z-spec:check` — verifies internal consistency of the specification
- **F3.** **Animation and model-checking with ProB** — `/z-spec:test` — explores the state space and finds invariant violations
- **F4.** **Automated tool installation** — `/z-spec:setup` and `/z-spec:doctor` — installs fuzz, ProB, and Lean; diagnoses environment issues
- **F5.** **Test case derivation from specs** — `/z-spec:partition` — applies the Test Template Framework to derive conformance test cases from the spec's mathematics, with optional executable code generation
- **F6.** **Runtime contracts** — `/z-spec:contracts` — generates precondition, postcondition, and invariant assertion functions from spec predicates, completing the test-coverage workflow

Should Do

Features that deepen the verification chain or extend the core workflows. Not required for the spec-first, test-coverage, or bug-finding loops but increase value. All items marked **shipped** are already available in v0.20.2.

- **F7.** **Code generation from specs** — `/z-spec:model2code` — generates implementation code and tests from a specification, completing the spec-first workflow. **Shipped.**
- **F8.** **Automated audit** — `/z-spec:audit` — runs the full model-check-test-report chain in one command for rapid bug finding across existing systems. **Shipped.**
- **F9.** **Lean 4 proof obligations** — `/z-spec:prove` — machine-checked proofs that invariants hold universally, not just for tested inputs. **Shipped.**
- **F10.** **Property-based test oracle** — `/z-spec:oracle` — uses Lean 4 model as ground truth for randomized testing. **Shipped.**
- **F11.** **Refinement verification** — `/z-spec:refine` — verifies that implementation code refines the specification via abstraction function commutativity. **Shipped.**
- **F12.** **B-Method support** — four commands, from `/z-spec:b-create` through `/z-spec:b-refine` — deterministic refinement chain from spec to provably correct code. **Shipped (alpha).**
- **F13.** **Spec elaboration** — `/z-spec:elaborate` — enhances a bare spec with narrative from adjacent design documentation, so the same file reads as both a mathematical model and a design brief. **Shipped.**
- **F14.** **Standalone CLI + MCP** — `z-spec` on `PATH` with no Claude Code plugin, installed via `-no-plugin`. All eleven deterministic verbs the plugin exposes — from `check` and `test` through `report` and `doctor` — are reachable from a shell, a script, or an agent that is not the Claude plugin (Codex, Cursor, a plain terminal). **Shipped.**

F15. **Tutor mode** — adaptive learning that teaches Z notation as the developer uses the tool, reducing the adoption barrier. **Open** (`z-spec-p4a`, `z-spec-yfv`).

Won't Do

Features explicitly excluded to maintain focus and identity.

F16. **Custom specification language** — Z is an ISO standard with established tooling. Inventing a new DSL would sacrifice the ecosystem (fuzz, ProB, textbooks, university courses) for marginal syntax improvement.

F17. **Visual specification editor** — Z Spec is a Claude Code plugin. The interaction model is conversational (natural language in, spec out), not graphical. A GUI would be a different product.

F18. **Runtime execution or deployment** — Specifications describe *what* a system does, not *how*. Z Spec generates artifacts (tests, contracts, proofs) that inform implementation but does not execute code or manage deployments.

References

[1] *Information technology — Z formal specification notation — Syntax, type system and semantics*. The international standard for Z notation. International Organization for Standardization, 2002.

[2] J. Michael Spivey. *The Z Notation: A Reference Manual*. 2nd. The definitive Z reference, by fuzz's creator. Prentice Hall, 1992. URL: https://spivey.oriel.ox.ac.uk/mike/zrm/.

[3] Solomon Lemma Abebe. *Specification-Driven Development: Rethinking How We Build Software in the Age of AI*. AIWare 2026. Three-level taxonomy: spec-first, spec-anchored, spec-as-source. Positions formal specs as primary artifact in AI-assisted development. 2026. URL: https://arxiv.org/abs/2602.00180.

[4] Jim Woodcock et al. "Formal methods: Practice and experience". In: *ACM Computing Surveys* 41.4 (2009). Survey of 62 industrial FM projects: 92% reported quality increases, 0% reported decreases., 19:1–19:36. DOI: 10.1145/1592434.1592436.

[5] Aaron Councilman et al. *Towards Formal Verification of LLM-Generated Code from Natural Language Prompts*. Formal query language as contract layer for LLM-generated Ansible code; verifier achieves 83% confirmation of correct code and 92% identification of incorrect code. 2025. URL: https://arxiv.org/abs/2507.13290.

[6] Suhas Bharadwaj et al. *Constitutional Spec-Driven Development: Enforcing Security by Construction in AI-Assisted Code Generation*. 73% reduction in security defects vs unconstrained AI generation when code is constrained by a formal spec layer. 2026. URL: https://arxiv.org/abs/2602.02584.

[7] Haolin Jin and Huaming Chen. "Uncovering Systematic Failures of LLMs in Verifying Code Against Natural Language Specifications". In: *ASE 2025: 40th IEEE/ACM International Conference on Automated Software Engineering*. LLMs exhibit systematic over-correction bias when verifying code. GPT-4o accuracy drops from 52.4% to 11.0% under complex prompts. 2025. URL: https://arxiv.org/abs/2508.12358.

[8] Stack Overflow. *Stack Overflow Developer Survey 2025*. 84% of developers use or plan to use AI coding tools, up from 76% in 2024. Only 29% trust the accuracy of AI-generated output. 2025. URL: https://survey.stackoverflow.co/2025/.

[9] Stack Overflow. *Stack Overflow Developer Survey 2024*. 76% of developers use or plan to use AI coding tools. 2024. URL: https://survey.stackoverflow.co/2024/.

[10] SlashData. *Global Developer Population Trends 2025*. Estimates 36.5 million professional developers within 47.2 million total developers worldwide as of early 2025. 2025. URL: https://www.slashdata.co/post/global-developer-population-trends-2025-how-many-developers-are-there.

[11] João Pascoal Faria et al. *Automatic Generation of Formal Specification and Verification Annotations Using LLMs and Test Oracles*. LLMs generate correct Dafny annotations for 98.2% of programs within 8 repair iterations using verifier feedback. 2026. URL: https://arxiv.org/abs/2601.12845.

[12] Martin Kleppmann. *Prediction: AI will make formal verification go mainstream*. LLM proof-script generation + formal checkers create a virtuous cycle. Cites seL4 (8,700 lines C, 20 person-years, 200,000 lines Isabelle) as evidence that cost barrier is collapsing. 2025. URL: https://martin.kleppmann.com/2025/12/08/ai-formal-verification.html (visited on 03/03/2026).

[13] Arshad Beg, Diarmuid O'Donoghue, and Rosemary Monahan. *Evaluating LLM-Generated ACSL Annotations for Formal Verification*. LLM-generated annotations show lower, more variable proof success rates and increased SMT solver instability vs tool-generated. 2026. URL: https://arxiv.org/abs/2602.13851.

[14] Qian Cheng et al. *SysMoBench: Evaluating AI on Formally Modeling Complex Real-World Systems*. LLMs handle small formal modeling artifacts; significant performance gaps remain for complex real-world distributed systems in TLA+. 2025. URL: https://arxiv.org/abs/2509.23130.

[15] Ye Liu et al. "PropertyGPT: LLM-driven Formal Verification of Smart Contracts through Retrieval-Augmented Property Generation". In: *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. 80% recall against human-written ground-truth properties; finds 26 known and 12 previously unknown vulnerabilities. 2025. URL: https://www.ndss-symposium.org/ndss-paper/propertygpt-llm-driven-formal-verification-of-smart-contracts-through-retrieval-augmented-property-generation/.

[16] Alessio Ferrari and Paola Spoletini. "Formal requirements engineering and large language models: A two-way roadmap". In: *Information and Software Technology* 181 (2025). Two symmetric paths: formal methods to guarantee LLM artifact correctness; LLMs to make formal methods more accessible., p. 107697. DOI: 10.1016/j.infsof.2025.107697.